

TECHNICAL DOCUMENTATION

ParkFlow

A futuristic, offline-first smart parking management platform — Flutter client, Node.js / Express API, and MongoDB.

PROJECT	ParkFlow · package park_ease	VERSION	1.0.0+1
FRONTEND	Flutter 3.x (Dart 3.11.5)	BACKEND	Node.js 18+ · Express 4 · Mongoose 8
DATABASE	MongoDB — 10 collections	HOSTING	Railway (API)
DOCUMENT	July 2026	SCOPE	Architecture · ER · DFD · Stack

- Offline-first
- Dual role · Driver / Operator
- QR ticketing
- UPI · INR pricing
- 144-slot garage sim

Contents

1	Project Overview	Purpose · features · garage model
2	Technology Stack	Frontend · backend · data · tooling
3	System Architecture	Four-layer offline-first design
4	Entity-Relationship Model	ER diagram · schema reference
5	Data Flow Diagrams	Context (L0) · decomposition (L1)
6	REST API Surface	Endpoints · auth · error model
7	Core Workflows	Reservation · validation · payment
8	Deployment, Security & Configuration	Railway · env · hardening notes
A	Appendix — Module & Source Map	Repository layout

1. Project Overview

What ParkFlow is, who uses it, and the garage it models.

ParkFlow is a full-stack, offline-first smart-parking application. It lets drivers discover free slots inside a multi-floor garage, reserve a slot, receive a QR-coded ticket, pay in INR through a UPI demo flow, and manage their booking history. A separate **operator** role lets parking staff scan and validate those tickets from the same app. The system is deliberately lightweight: state is handled by Flutter's built-in `ChangeNotifier`, offline resilience comes from a `shared_preferences` cache and a local simulation engine that keeps the UI alive with no network.

1.1 Roles

Role	Capabilities
Driver default	Register / sign in, manage a garage of vehicles, browse the live parking map, reserve · extend · cancel a slot, hold QR tickets, receive notifications, and review history & receipts.
Operator staff	Scan any driver's QR ticket, run it through the server-side validation state machine (valid / already-used / expired / invalid), keep a scan log, and view the analytics / admin dashboard.

1.2 Key features

- Multi-floor interactive parking map with real-time slot colour-coding, drawn with a custom painter.
- **Offline-first** architecture — the local cache survives network outages and cold starts.
- A simulation engine that nudges slot availability every 3 seconds for demo realism (never persisted).
- QR-coded tickets generated on the client and validated server-side across users.
- Full booking lifecycle: reserve → extend → cancel, with fee accrual.
- UPI demo payment flow with Indian-numbering INR pricing at **₹40 / hour**.
- Push-style in-app notifications, persisted per user on the server.
- Light / dark theming around a futuristic brand-green (#22C55E) palette.

1.3 Garage model

Parameter	Value
Layout	4 floors (F1 – F4) × 36 per floor (6 × 6 grid) = 144 slots
Slot label format	F<floor>-<rowLetter><col> · e.g. F2-C4
Slot states	available · occupied · reserved · disabled
Pricing	₹40 / hour — accrued in minor units (paise / cents)
Simulation tick	every 3 s, local only (not written to the server)

2. Technology Stack

Every runtime, framework and library the project depends on — and why.

2.1 Frontend — Flutter client (package park_ease, ~11,000 lines of Dart)

Technology	Version	Purpose
Flutter / Dart	SDK ^3.11.5	Cross-platform UI for Android, iOS and Web from one codebase.
State management	ChangeNotifier	Built-in single-source-of-truth AppState exposed via an InheritedNotifier — no third-party state package.
http	^1.2.0	Thin REST client to the backend (bearer token, JSON, 20 s timeout).
shared_preferences	^2.3.2	Offline key/value cache — auth token, settings, mirrored collections.
qr_flutter	^4.1.0	Renders the QR code printed on each parking ticket.
mobile_scanner	^7.2.0	Camera-based QR reader used by operators.
intl	^0.20.2	Date / time and number formatting.
google_fonts	^6.2.1	Typography for the branded theme.
cupertino_icons	^1.0.8	iOS-style icon set.
flutter_lints	^6.0.0	Static-analysis rule set (dev dependency).

2.2 Backend — Node.js / Express API (~1,160 lines of JavaScript, CommonJS)

Technology	Version	Purpose
Node.js	>= 18	Server runtime (declared in engines).
Express	^4.19.2	HTTP routing and the middleware pipeline (CORS, JSON, auth, errors).
Mongoose	^8.5.0	MongoDB ODM — schemas, validation, indexes, JSON transforms.
dotenv	^16.4.5	Loads PORT and MONGODB_URI from the environment.
CORS	hand-rolled	A tiny inline middleware (no dependency) that opens the API to the web client.

2.3 Data & infrastructure

Layer	Choice	Notes
-------	--------	-------

Database	MongoDB	10 collections; per-user scoping and unique compound indexes.
API hosting	Railway	Config in <code>railway.json</code> ; <code>/health</code> probe with no DB dependency.
Client targets	Android · iOS · Web	Base URL overridable at build time via <code>--dart-define=API_BASE_URL</code> .
Money	INR (paise)	Integer minor units; hand-rolled Indian-grouping formatter (₹1,25,000).

Design intent — keep the dependency surface tiny

State management, CORS, the money formatter and the PIN hash are all built in-house rather than pulled from packages. The result is a codebase that is easy to reason about and cheap to deploy, at the cost of a few demo-grade simplifications called out in Section 8.

3. System Architecture

A four-layer, offline-first design with a stateless API in the middle.

ParkFlow is organised into four cooperating layers. The Flutter client is split into a **presentation** layer and a **client data & state** layer; the server is a stateless **Express** application layer over a MongoDB **data** layer. The client talks to the server over plain JSON REST with a bearer token, and the server talks to the database through Mongoose. Crucially, every screen reads from an in-memory cache that is mirrored to device storage, so the UI is fully interactive before — or without — any server round-trip.

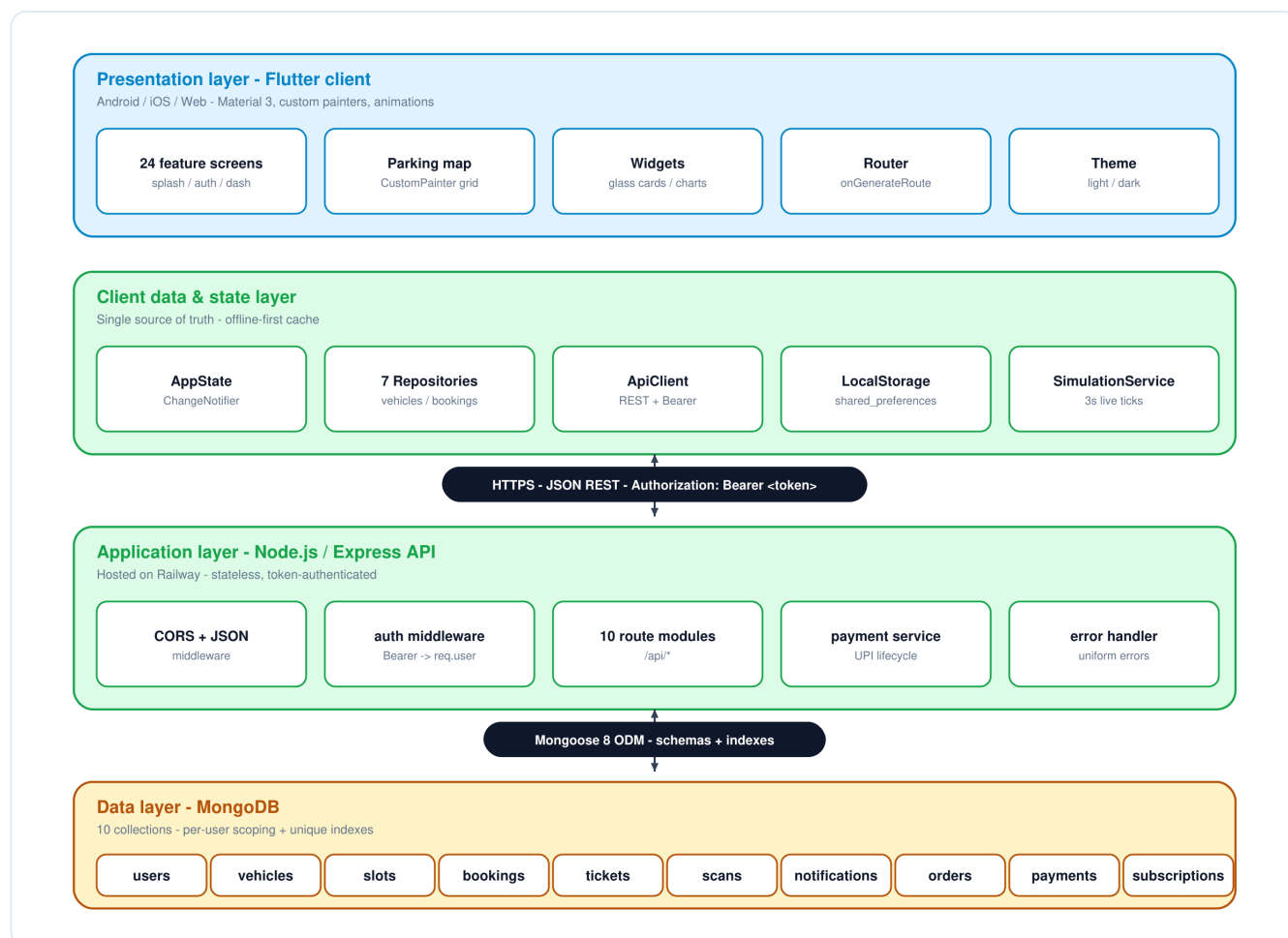


Figure 3.1 — Layered architecture. The client caches locally and syncs opportunistically; the API is stateless and token-authenticated.

3.1 Client layers

The presentation layer holds 24 feature screens (splash, onboarding, auth, dashboard, parking map, booking, tickets, scanner, vehicles, history, analytics, notifications, settings and the admin shell), a custom-painter parking grid, reusable widgets, an `onGenerateRoute` router, and the themed design system. Beneath it, `AppState` — a single `ChangeNotifier` — is the source of truth. It composes seven repositories (auth, vehicles, parking, bookings, tickets, scans, notifications), each of which owns a synchronous in-memory cache, mirrors that cache to `shared_preferences`, and lazily syncs with the API through the shared `ApiClient`. A `SimulationService` ticks every three seconds to animate slot occupancy locally.

3.2 Backend layers

The Express app wires a minimal CORS shim and JSON body-parser, a bearer-token auth middleware that resolves a token to `req.user`, ten route modules mounted under `/api/*`, a payment service that owns the UPI state machine, and a centralised error handler that gives every failure the same JSON shape. Models are Mongoose schemas; most application collections are built by a shared `clientSchema` factory that adds a client-supplied `id`, optional per-user scoping, and a `toJSON` transform so the payload matches the Flutter model exactly.

3.3 How a request flows

On sign-in the client stores the returned token and calls `_pullAll()`, hydrating all repositories in parallel. Each subsequent mutation (add a vehicle, reserve a slot) writes through to the server and updates the local cache. Reads render instantly from cache; a background pull reconciles with the server when reachable.

Offline-first guarantee

Every repository pull is wrapped in a `_guard()` that swallows network errors and keeps the last-known cached data. The server also binds its port *before* connecting to MongoDB, so the Railway health check passes and clients receive a real error instead of a 502 when the database is briefly unreachable.

4. Entity-Relationship Model

Ten MongoDB collections across two modelling styles.

ParkFlow's data divides into two families. The **application collections** mirror the Flutter models one-to-one: each document carries a client-supplied string `id` and (except the shared garage layout) a `userId` that scopes it to its owner. The **commerce collections** — orders, payments and subscriptions — are server-native: they use Mongo's `ObjectId`, carry `createdAt` / `updatedAt` timestamps, and store money as integer `paise`.

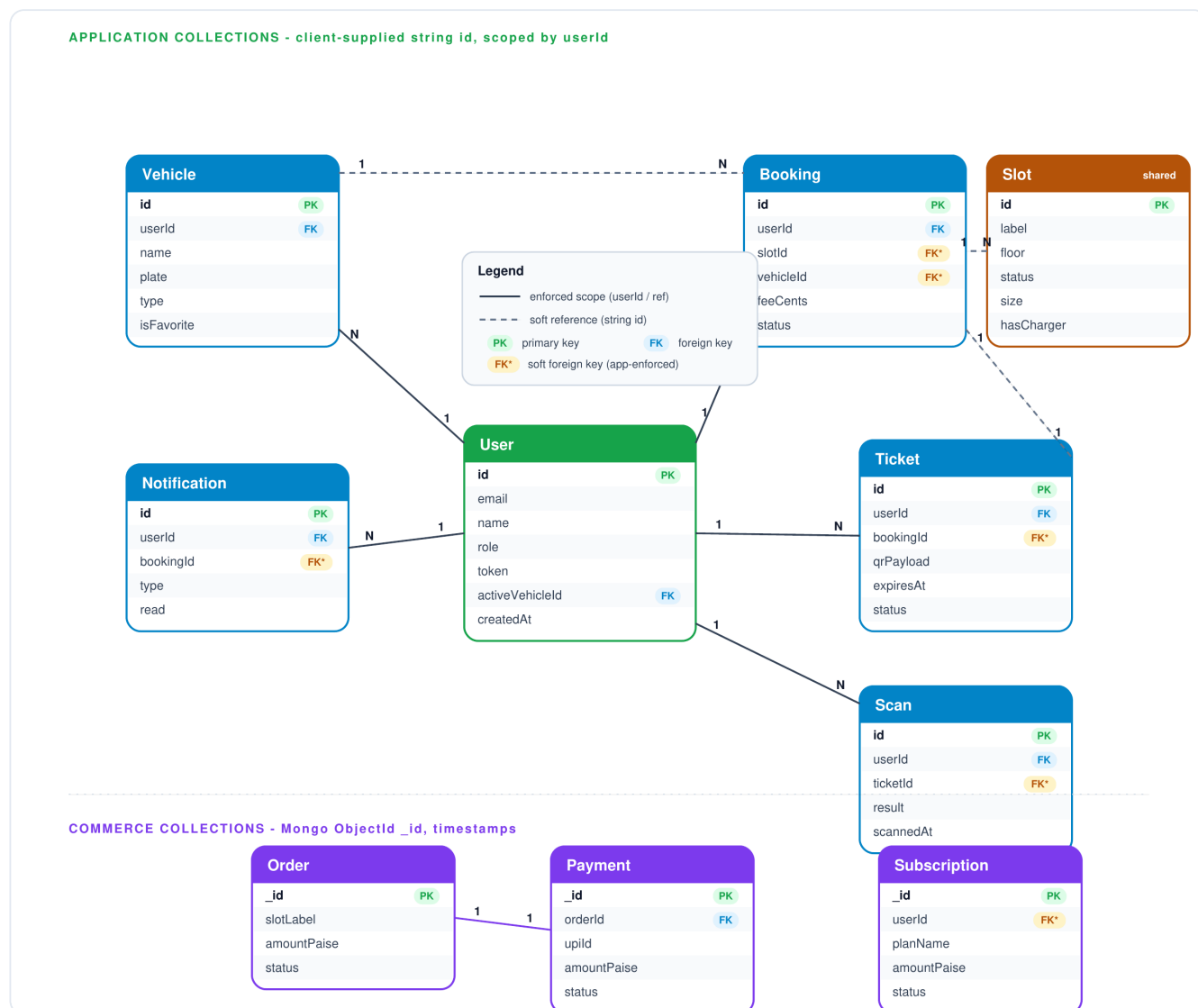


Figure 4.1 — Entity-relationship diagram. Solid links are enforced scopes / references; dashed links are soft (string-`id`) references resolved in application code.

4.1 Two modelling styles

Aspect	Application collections	Commerce collections
Members	User, Vehicle, Slot, Booking, Ticket, Scan, Notification	Order, Payment, Subscription
Primary key	<code>id</code> — client string	<code>_id</code> — Mongo <code>ObjectId</code>

Scoping	Required <code>userId</code> + unique (<code>userId</code> , <code>id</code>) index (Slot is shared / unscoped)	Not user-scoped; referenced by <code>ObjectId</code>
References	Soft string ids, enforced in app code	<code>Payment.orderId</code> is a real Mongoose ref with a unique partial index
JSON	<code>toJSON</code> strips <code>_id</code> / <code>userId</code> to match the client model	<code>toJSON</code> adds <code>amount</code> / <code>formattedAmount</code> (₹)

4.2 Collection reference

Field-level schema for each collection. PK primary key · FK reference · types as declared in Mongoose.

USER — account & session

Field	Type	Constraints / notes
<code>id</code>	String	PK required, unique.
<code>name</code>	String	required.
<code>email</code>	String	required, unique, lowercased & trimmed.
<code>passwordHash</code>	String	SHA-256 of a prefixed password; blanked in JSON output.
<code>role</code>	String (enum)	<code>driver</code> <code>operator</code> · default <code>driver</code> .
<code>token</code>	String	indexed bearer token; stripped from JSON.
<code>activeVehicleId</code>	String	FK soft → <code>Vehicle</code> .
<code>avatarSeed</code>	Number	deterministic avatar seed · <code>createdAt</code> Date.

VEHICLE — scoped

Field	Type
<code>id</code>	String PK
<code>userId</code>	String FK
<code>name / plate</code>	String, required
<code>type</code>	String (<code>car</code> / <code>ev</code> ...)
<code>colorValue</code>	Number, required
<code>notes</code>	String
<code>isFavorite</code>	Boolean

SLOT — shared / unscoped

Field	Type
<code>id</code>	String PK
<code>floor / row / col</code>	Number, required
<code>label</code>	String, required
<code>status</code>	String · def. <code>available</code>
<code>size</code>	String · def. <code>standard</code>
<code>hasCharger</code>	Boolean
<code>walkingDistance / score</code>	Number

BOOKING — scoped

Field	Type
id	String PK
userId	String FK
slotId / slotLabel	String FK
vehicleId	String FK
floor	Number
startTime / endTime	Date, required
feeCents	Number, required
status	String · def. active

TICKET — scoped

Field	Type
id	String PK
userId	String FK
bookingId	String FK
slotLabel / floor	String / Number
vehiclePlate	String
qrPayload	String, required
issuedAt / expiresAt	Date, required
status	active → used / expired

SCAN — scoped, operator log

Field	Type
id	String PK
userId	String FK
result	String, required
scannedAt	Date, required
ticketId	String FK
slotLabel / vehiclePlate	String
rawCode	String

NOTIFICATION — scoped

Field	Type
id	String PK
userId	String FK
title / body	String, required
type	String · def. system
createdAt	Date, required
read	Boolean
bookingId	String FK

COMMERCE — ORDER · PAYMENT · SUBSCRIPTION (ObjectId, timestamps, amountPaise)

Collection	Key fields
Order	_id, slotLabel, floor, vehiclePlate, startTime, endTime, amountPaise, currency, cancelledAt
Payment	_id, orderId FK→Order , amountPaise, upiId, qrData, cancelledAt
Subscription	_id, userId, planName, amountPaise, renewsAt, cancelledAt

4.3 Relationship summary

Relationship	Cardinality	Via	Enforcement
User → Vehicle	1 : N	userId	required + unique (userId, id)
User → Booking / Ticket / Scan / Notification	1 : N	userId	required + unique (userId, id)
User → activeVehicle	1 : 1	activeVehicleId	soft (app code)
Vehicle → Booking	1 : N	vehicleId	soft (string id)
Slot → Booking	1 : N	slotId	soft (string id)
Booking → Ticket	1 : 1	bookingId	soft (string id)
Ticket → Scan	1 : N	ticketId	soft (string id)
Booking → Notification	1 : N (opt.)	bookingId	soft (string id)
Order → Payment	1 : 1	orderId	hard ObjectId ref + unique partial index (one live payment / order)
User → Subscription	1 : N	userId	soft (string id)

5. Data Flow Diagrams

How data moves between the people, the processes and the stores.

5.1 Context diagram (Level 0)

At the highest level ParkFlow is a single process serving two human actors — the **Driver** and the **Operator** — and one external service, the **UPI / payment provider** (a demo integration). Drivers send profile, reservation and payment details and receive slot availability, QR tickets and alerts; operators submit scanned QR codes and receive validation results; the payment provider receives a UPI intent and returns a success / failed webhook.

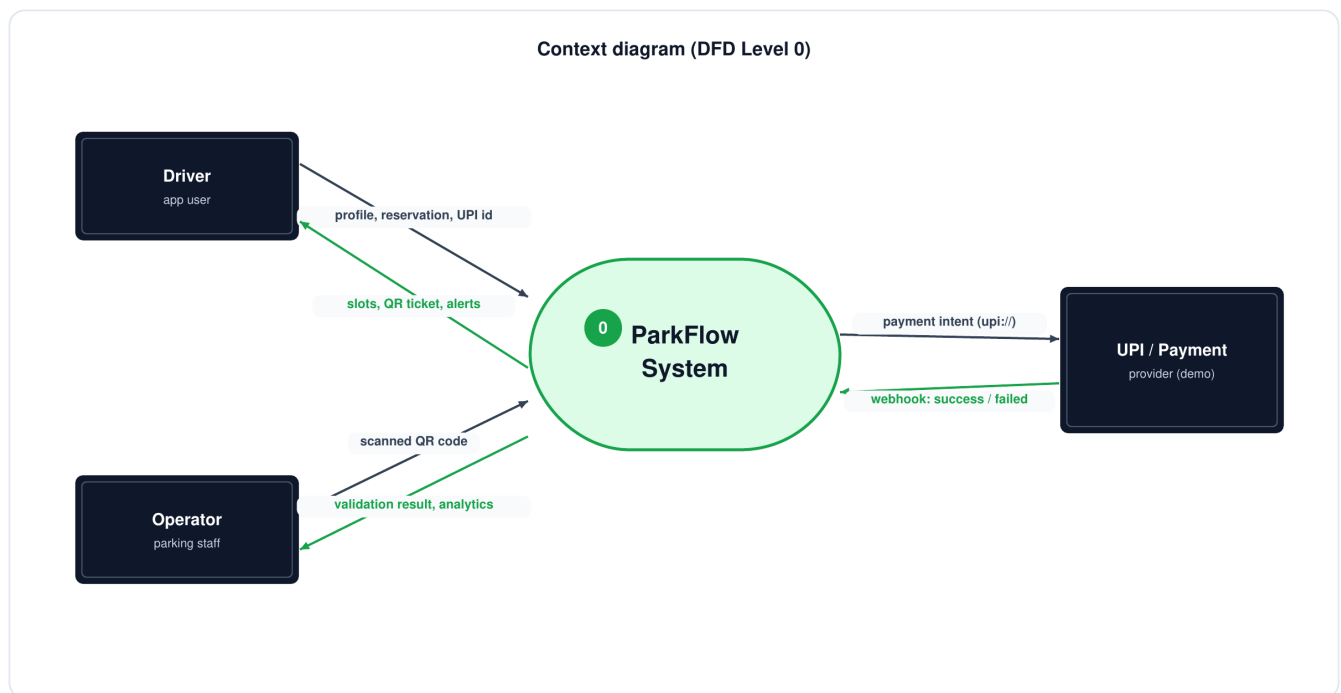


Figure 5.1 — DFD Level 0 (context). One system boundary, two human actors, one external payment service.

External entity	Sends to system	Receives from system
Driver	credentials, vehicle details, reservation request, UPI id	slot availability, QR ticket, notifications, receipt
Operator	scanned QR code	validation result, scan history, analytics
UPI / PSP	payment webhook (success / failed)	payment intent (upi : // QR string)

5.2 Level-1 decomposition

Exploding the single process reveals seven sub-processes, each backed by one or more of the ten data stores. Processes 1 – 6 map to the token-authenticated `/api` resources; process 7 covers the commerce routes (orders, payments, subscriptions).

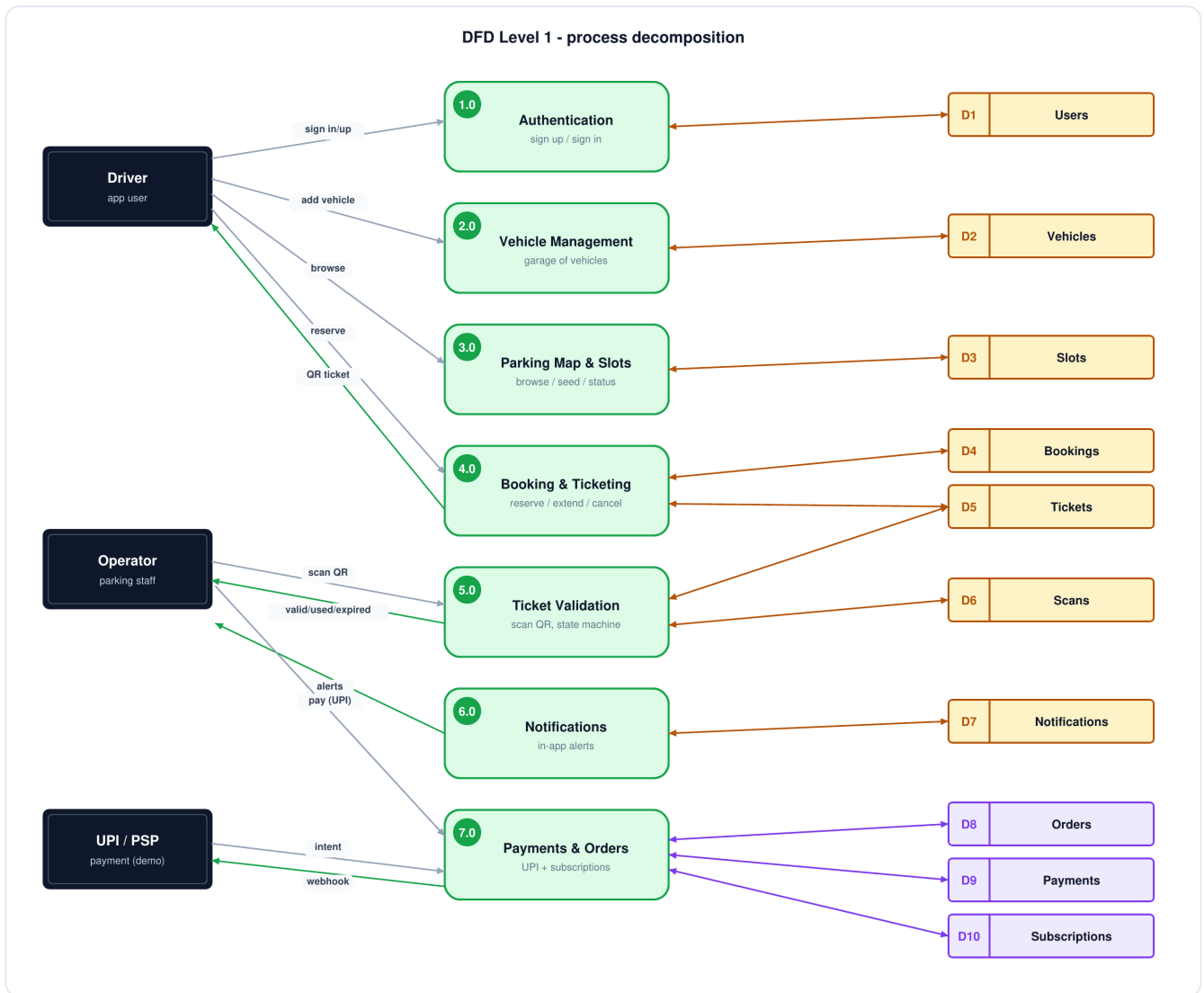


Figure 5.2 — DFD Level 1. Amber stores (D1–D7) are application collections; violet stores (D8–D10) are commerce collections.

#	Process	Primary flows	Data stores
1.0	Authentication	sign-up / sign-in; issues bearer token	D1 Users
2.0	Vehicle Management	add / edit / delete; set active vehicle	D2 Vehicles
3.0	Parking Map & Slots	seed layout, browse, update status	D3 Slots
4.0	Booking & Ticketing	reserve / extend / cancel; mint QR ticket	D4 Bookings · D5 Tickets
5.0	Ticket Validation	scan QR, run state machine, log scan	D5 Tickets · D6 Scans
6.0	Notifications	create, list, mark read	D7 Notifications
7.0	Payments & Orders	order → UPI payment → settle; subscriptions	D8 Orders · D9 Payments · D10 Subs

Reading the stores

All application stores (D1–D7) are read-and-write per authenticated user. The commerce stores (D8–D10) are keyed by ObjectId and are, in this build, reachable without a token — see the hardening notes in Section 8.

6. REST API Surface

Every endpoint the Flutter client calls, grouped by resource.

The API is a stateless Express application. Public endpoints cover authentication and the commerce resources; everything under a user's own data requires an Authorization: Bearer <token> header, which the auth middleware resolves to req.user.

6.1 Authentication public

Method	Path	Purpose
POST	/api/auth/signup	Create an account; returns { user, token }.
POST	/api/auth/signin	Authenticate; returns { user, token }.

6.2 Vehicles · Slots · Bookings bearer

Method	Path	Purpose
GET	/api/vehicles	List vehicles + activeVehicleId.
POST	/api/vehicles	Upsert a vehicle by id (first becomes active).
PUT	/api/vehicles/active	Set the active vehicle.
PUT	/api/vehicles/:id	Update a vehicle.
DEL	/api/vehicles/:id	Delete; re-points active vehicle if needed.
GET	/api/slots	Return the garage; seeds 144 slots on first call.
PATCH	/api/slots/:id/status	Set a slot's status (reserve / free).
GET	/api/bookings	List the user's bookings.
POST	/api/bookings	Upsert a booking by id.
PATCH	/api/bookings/:id	Cancel or extend a booking.

6.3 Tickets · Scans · Notifications bearer

Method	Path	Purpose
GET	/api/tickets	List the user's tickets.
POST	/api/tickets/validate	Validate a scanned code across all users; runs the ticket state machine.
POST	/api/tickets	Upsert a ticket by id.
PATCH	/api/tickets/:id	Update ticket status (used / expired).

GET	/api/scans	Scan history, newest first.
POST	/api/scans	Record a scan event.
DEL	/api/scans	Clear the user's scan history.
GET	/api/notifications	List notifications, newest first.
POST	/api/notifications	Upsert a notification.
PATCH	/api/notifications/read-all	Mark all as read.
PATCH	/api/notifications/:id/read	Mark one as read.

6.4 Commerce — Orders · Payments · Subscriptions public

Method	Path	Purpose
POST	/api/orders	Create a parking order.
GET	/api/orders/:id	Fetch an order.
POST	/api/orders/:id/cancel	Cancel (idempotent).
POST	/api/payments/create	Create a pending UPI payment; returns qrData.
POST	/api/payments/success · /fail	Settle a pending payment to a terminal state.
POST	/api/payments/webhook	PSP callback simulation (success / failed).
POST	/api/payments/:id/cancel	Cancel a payment (success cannot be cancelled).
GET	/api/payments/:id	Fetch a payment.
POST	/api/subscriptions · /:id/renew · /:id/cancel	Create, renew (blocked once cancelled) or cancel a pass.
GET	/api/subscriptions/:id	Fetch a subscription.
GET	/health	Liveness probe (no DB dependency).

6.5 Auth & error model

Successful responses return the resource JSON. Failures share one shape: { success:false, message, errorCode }. Mongoose ValidationError maps to HTTP 422 (VALIDATION_ERROR), a bad ObjectId to 400 (INVALID_ID), and any 5xx is masked to a generic message in production. Representative codes include UNAUTHORIZED, EMAIL_TAKEN, INVALID_CREDENTIALS, DUPLICATE_PAYMENT, INVALID_UPI and NOT_FOUND.

7. Core Workflows

The three flows that define the product.

7.1 Reservation → ticket

Reservation is orchestrated on the client and written through to the server as it happens:

Step	Action
1	User picks a slot and duration. <code>AppState.reserveSlot()</code> computes the fee as ₹40 × hours, stored as <code>feeCents</code> .
2	A Booking and a Ticket are built locally. The QR payload is <code>PARKFLOW bookingId slotLabel plate expiryMs</code> .
3	The booking and ticket are POSTed (upsert by id); the slot status is PATCHed to reserved.
4	A confirmation Notification is created, and the UI navigates to the success screen with the live QR.

7.2 Ticket validation state machine

Validation runs server-side so an operator can scan any driver's pass. `/api/tickets/validate` looks the code up by `qrPayload` (falling back to the `bookingId` embedded in the payload) and applies:

Condition	Result	Effect on ticket
No ticket found	<code>invalid</code>	<code>none</code>
Status already used	<code>alreadyUsed</code>	<code>none</code>
Past <code>expiresAt</code> or expired	<code>expired</code>	<code>status → expired (persisted)</code>
Otherwise	<code>valid</code>	<code>status → used (persisted)</code>

The client then logs the outcome as a **Scan** event.

7.3 UPI payment lifecycle

Step	Action
1	An Order is created for the parking charge (amount in paise, currency INR).
2	<code>/api/payments/create</code> validates the UPI id, builds a <code>upi://pay?...</code> QR string, and creates a pending payment. A unique partial index allows at most one live payment per order.
3	The payment settles to success or failed via the success / fail endpoints or the webhook simulation.
4	Cancellation is idempotent; a success payment can never be cancelled. Amounts are returned pre-formatted (e.g. ₹1,25,000) by the money helper.

8. Deployment, Security & Configuration

How it runs, how it authenticates, and what to harden before production.

8.1 Environment & deployment

Variable	Role
PORT	Listen port (default 5000). Bound before the DB connects so the health check stays green.
MONGODB_URI	MongoDB connection string (required; the app throws if missing).
NODE_ENV	production masks internal 5xx messages.
API_BASE_URL	Client-side build flag (<code>--dart-define</code>) pointing the Flutter app at the API.

The backend deploys to **Railway** (configuration in `railway.json`) and is reached by the client at its production base URL; `GET /health` is the platform liveness probe.

8.2 Security model

- **Server auth** — an opaque 24-byte hex bearer token is minted at sign-up / sign-in, stored on the user, and resolved to `req.user` on every protected call.
- **Passwords** — hashed with SHA-256 over a versioned prefix; the hash is never returned (blanked in `toJson`).
- **Device app-lock** — the PIN and biometric toggle are stored and verified locally on the device (a separate lightweight hash) and are independent of server authentication.

Hardening checklist before production

This build favours demo simplicity. Before a real deployment, consider: salted / adaptive password hashing (bcrypt or argon2) instead of plain SHA-256; expiring, rotatable tokens (or JWT) rather than a permanent token; authentication on the commerce routes (orders / payments / subscriptions are currently open); and a tightened CORS origin in place of the wildcard `*`. The 3-second slot simulation is intentionally client-only and never written to the database.

Appendix A — Module & source map

FRONTEND/LIB

Path	Contents
app/	AppState, router, root widget
core/	constants, theme, storage
data/models/	7 client models
data/repositories/	7 server-backed repos
data/api/	ApiClient (REST)
data/services/	SimulationService
features/	24 feature screens
widgets/	shared UI components

BACKEND/SRC

Path	Contents
server.js	app wiring & boot
db.js	Mongoose connection
middleware/	auth, error handler
models/	10 schemas + _doc factory
routes/	10 route modules
services/	payment lifecycle
utils/	money, asyncHandler

End of document · ParkFlow v1.0.0+1 · July 2026